



Forensic analysis of B-tree file system (Btrfs)

Wasim Ahmad Bhat*, Mohamad Ahtisham Wani

Department of Computer Sciences, University of Kashmir, India

ARTICLE INFO

Article history:

Received 4 May 2018

Received in revised form

3 August 2018

Accepted 4 September 2018

Available online 14 September 2018

Keywords:

Btrfs

B-Tree

Forensic artifacts

Evidence-extraction

Node-balancing

ABSTRACT

This paper identifies forensically important artifacts of B-tree file system (Btrfs), analyses changes that they incur due to node-balancing during file and directory operations, and based on the observed file system state-change proposes an evidence-extraction procedure. The findings suggested that retrieving forensic evidence in a fresh B-tree file system is difficult, the probability of evidence-extraction increases as the file system ages, internal nodes are the richest sources of forensic data, degree of evidence-extraction depends upon whether nodes are merged or redistributed, files with size less than 1 KB and greater than 4 KB have highest chances of recovery, and files with size 3–4 KB have least chances of recovery.

© 2018 Elsevier Ltd. All rights reserved.

Introduction

B-tree file system (Btrfs) (Rodeh et al., 2013) is the latest and the most advanced file system offered by Linux. To cope up with diverse, intense and sensitive workloads, Btrfs offers a paradigm shift in file system design by providing simple administration, end-to-end data integrity, and immense scalability without loss of performance. Btrfs is expected to work well on systems as small as a smartphone and as large as enterprise production servers. This puts Btrfs at forefront in delivering complete and reliable forensic information in case of a breach or an intrusion. Another compelling reason for witnessing a strong relationship between computer crime and Btrfs is the lack of literature that unearths its vulnerabilities, and the weaknesses of present digital forensic techniques (Newsham et al., 2007).

In this paper, we identify forensically important artifacts of Btrfs, analyse the changes that they incur due to node-balancing during file and directory operations, propose an evidence-extraction procedure based on the observed file system state-change, and discuss the results. This work is the first of its kind that details the forensic capabilities of Btrfs.

The rest of this paper is organized as follows. Section 2 discusses the related work, section 3 identifies various Btrfs artifacts and highlights their forensic importance, section 4 analyses the impact

of node *splitting*, *redistribution* and *merging* on the identified forensic artifacts during file and directory operations, and section 5 presents the proposed evidence-extraction procedure based on the forensic artifacts identified and the observed file system state-change due to node *splitting*, *redistribution* and *merging*. Section 6 discusses the results, section 7 highlights various challenges Btrfs offers to forensic investigators, and section 8 discusses the conclusion.

Related work

Forensic analysis of Linux file systems (and other file systems that are related to Btrfs in design) point towards techniques that retrieve forensic evidence by analysing various file system data structures and changes that they incur during file and directory operations, mainly deletion operation. One work describes how data hidden in the reserved portions of the Ext2/Ext3 file systems can be detected using data detection utility and file system checkers by exploiting file system data structures (Piper et al., 2005). Another work (Barik et al., 2007) describes how existing last Modified, last Accessed, and Creation Date Time Stamp (MAC-DTS) procedures are inherently flawed and inconsistent in providing a reliable timeline for digital evidence in context of Ext2 file system, and argues how Authentic Date Time Stamps (ADTS) could solve it. Similarly, another work (Chen and Liu, 2010) proposes an object-oriented method of analysing Ext2 file system by creating an object model of different data structures of the file system.

While there are several papers that document the design of Ext4 file system, only few present Ext4 file system from the perspective

* Corresponding author.

E-mail addresses: wab.cs@uok.edu.in (W.A. Bhat), ahtishamwani@gmail.com (M.A. Wani).

of digital forensics. One such work (Fairbanks, 2012) provides a comprehensive analysis of Ext4 file system with the forensic community in mind and states some brief implications of the file system behaviour with respect to data recovery. The work draws attention to forensically important artifacts of Ext4 file system such as increased timestamp resolution, persistent preallocation, block group layout, and so on.

XFS file system is one of the file systems specialized for servers and designed to process huge files. The forensic analysis of XFS file system (Park et al., 2015) examines how file system data structures can be used to recover deleted files which can aid investigators in forensic investigation. It is based on the same technique as used for Ext2 file system (Chen and Liu, 2010).

ZFS file system is different in its data structures and disk layout but similar to Btrfs due to its copy-on-write (COW) transactional object model, snapshots, and clones. The work described in (Beebe et al., 2009) discusses the ZFS forensics in context of ZFS architecture, functionality, disk layout, and identifies various artifacts left behind by normal ZFS operation. These artifacts are of forensic nature particularly in time-line reconstruction of events, and are recoverable during analysis.

BTRFS forensic artifacts

Forensic analysis of the file system begins with identification of key data structures that have the potential to become evidence sources. These forensic artifacts form a bedrock over which the entire investigation can be sustained and presented in the court of law. In this section, we identify many such key digital forensic artifacts that exist on Btrfs, and analyse them from a forensic point of view.

Superblock

The Superblock is the first key-element that is important to file system forensic analysis as it points towards other important data structures of the file system just like \$Boot file does in NTFS (Nagar, 1997). In Btrfs, the Superblock mirrors are kept at fixed locations: 64 KB into every block device, with redundant copies (for reliability) at 64 MB, 256 GB and 1 PB. The Superblock contains many fields, and those which are forensically important are shown in Fig. 1. Note that the values stored in these fields are expressed in little-endian format.

Among these fields, some fields indicate the type, version, size, and other features of the file system. The field `sb_MagicNum` is the file system identifier, and is necessary to confirm whether the file system is Btrfs or not. The field `sb_Generation` contains Superblock generation number, which is a monotonically increasing number recorded during a checkpoint. These checkpoints by default occur after every 30 seconds (Rodeh et al., 2013). As Superblock is always edited in-place, the `sb_Generation` can be used to anticipate the version of the Superblock, or how many times the Superblock has been updated. The field `sb_TotalBytes` indicates the total number of bytes that the file system contains;

the field `sb_BytesUsed` refers to the bytes in use; and the field `sb_SectorSize` specifies the sector size. Btrfs stores data and metadata in tree nodes, and their size is identified by the fields `sb_NodeSize` and `sb_LeafSize`. The field `sb_NodeSize` contains the tree block size, whose default value is 4 KB, and can support a maximum value of 64 KB. The field `sb_LeafSize` contains the size of leaf nodes and always equals node size. Besides these fields, the field named `sb_RootTreeAddr` contains the starting logical-address of the *root-tree* root. This field in combination with the field `sb_NodeSize` is used to retrieve the root node of the *root-tree* (explained in section 3.2). Also, another field `sb_RootDirObjId` contains the *object-id* of root directory, which is usually 6. All these fields are forensically important as they not only identify the file system type and version, but also provide all the necessary data (like address and size) to locate all other data structures of the file system.

Root-tree

The Btrfs *root-tree* is a special B-tree that indexes all other B-trees making up the file system. It is also known as the *tree of tree-roots* as it acts as a directory of all other trees on the file system. And as already mentioned, the Superblock points to the *root-tree*'s root node. Furthermore, clones and snapshots created on the file system are also indexed by the *root-tree*. The *root-tree* in Btrfs performs the same function as does the Master File Table (\$MFT) in NTFS (Bhat and Quadri, 2011), i.e., identifying other system structures.

The Btrfs makes use of copy-on-write (COW) method to make changes in the file system. The COW technique loads the data from disk to memory, makes necessary modifications and finally writes the modified data to a different disk location. The idea is to ensure blocks and *extents* are not overwritten in-place to avoid file system corruption in case of a power failure or a partial update (Rodeh et al., 2013). This method also allows Btrfs to facilitate wear-leveling in case of solid-state drives (SSDs). However, due to COW updates, all changes in the file system ripple up and are finally captured at the top most level as a new root in the *root-tree*. This implies that the *root-tree* holds the same forensic importance as that of the Superblock in the file system given the magnitude at which the file system trees are COWed and checkpointed (Rodeh et al., 2013).

Btrfs B-trees

B-tree is a familiar concept and its detailed description can be found in (Comer, 1979) (Rodeh, 2008) (Rodeh, 2010). Btrfs on-disk layout is a forest of such B-trees. These trees include *subvolume-tree* that stores user visible files and directories, *extent-tree* for allocated *extents*, *checksum-tree* for storing checksums per allocated *extent*, and *chunk* and *device-trees* for mirroring, striping and RAID implementation. These trees are used by Btrfs to store data and metadata information of the file system. This makes them similar to the \$MFT file of NTFS (Bhat and Quadri, 2011). A B-tree of order m (where m is the maximum number of keys/elements that a node can store) is a self balancing tree subject to the constraint that each leaf node is at least half full (i.e., contains atleast $m/2$ keys) and each internal node contains at least $(m/2) - 1$ keys. Any change in this required configuration imbalances the tree leading to invocation of balancing algorithms. Since leaf nodes of Btrfs B-trees contain data elements and internal nodes contain pointers, any insertion or removal of data from leaf nodes may imbalance the tree. In order to re-balance the tree, these balancing algorithms either split the nodes (during insertion), or merge/redistribute them (during deletion). Therefore, node-balancing becomes fundamental part of B-tree functionality, and this section provides a comprehensive working of these node-balancing algorithms. For further details, interested

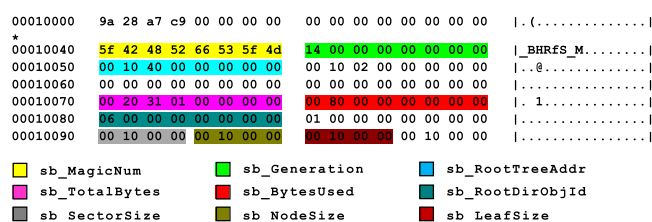


Fig. 1. Btrfs Superblock fields necessary for forensic analysis.

readers are referred to (Comer, 1979) (Rodeh, 2008).

Algorithm 1 shows all the necessary functions and steps of *node-splitting* during insertion of a new element. In case of insertion, first an appropriate leaf is identified for the element based on its key (identified by `FindLeaf()` function). The identified leaf is then checked for necessary available space required by the element (checked by `Overflow()` function). If the leaf has enough space and does not result in overflow of elements, the element is simply added to the node (using `AddData()` function). However, if the identified leaf is full, the space required by the new element is carved through *node-splitting* (using `Split()` function). In `Split()` function, a new node is created (using `CreateNew()` function) and elements from the original leaf along with the new element are equally distributed (using `Balance()` function) among the two nodes (i.e., original and new node). These leaf nodes are then updated to point to their neighbouring leaf nodes (using `UpdateSiblings()` function) and their parent (using `SetParent()` function). Finally, the parent is also updated to point towards both nodes (using `PushKeyUp()` function). However, if the parent is already full, the same splitting process to balance the internal node (i.e., the parent) begins (in `PushKeyUp()` function) and may ripple up possibly reaching upto the root where it is finally captured by creating a new root, thereby increasing the height of the B-tree by one.

Algorithm 1 Algorithm for splitting of nodes on insertion

Require: *Key, value*

```

1: function INSERT(key, value)
2:   leaf ← FINDLEAF(key)
3:   if OVERFLOW(leaf) then
4:     SPLIT(leaf)
5:   end if
6:   ADDDATA(leaf, key, value)
7: end function
8: function SPLIT(node)
9:   n ← CREATENEW
10:  mid ← node.keyCount/2
11:  upKey ← node.key[mid]
12:  BALANCE(mid, node, n)
13:  UPDATESIBLINGS(n, node)
14:  p ← GETPARENT(node)
15:  SETPARENT(n, p)
16:  PUSHKEYUP(p, upKey)
17: end function
18: function PUSHKEYUP(int, key)
19:   if int != root then
20:     node ← FINDINTERNAL(int)
21:     if OVERFLOW(int) then
22:       SPLIT(int)
23:     end if
24:     ADDDATA(int, key)
25:   else
26:     n ← CREATENEW
27:     UPDATESIBLINGS(n, root)
28:     SETPARENT(n, root)
29:     root = n
30:     ADDDATA(n, key)
31:   end if
32: end function

```

Algorithm 2 shows all the necessary functions and steps of *node*

-redistribution and *node-merging* during deletion of an existing element. In case of deletion, the element is simply removed (by `RemoveElement()` function) once the leaf containing the element is identified (by `FindLeaf()` function). If the removal breaks node-balancing constraint for the leaf (checked by `UnderFlow()` function), the elements of the leaf and its neighbours are either redistributed among them or the leaf is merged with one of its neighbours (both accomplished in function `DealUnderFlow()`) so that the constraint is satisfied. The `DealUnderFlow()` function first checks if the leaf becomes empty (checked by `IsEmpty()` function), in which case its key is removed from its parent (using `RemoveElement()` function). However, if the parent also underflows (checked by `UnderFlow()` function), the same *redistribution* or *merging* process to balance the internal node (i.e., the parent) begins, thereby triggering a ripple effect all the way up to the root (by making recursive call to `DealUnderFlow()` function). In other case, the neighbours of the node are checked whether the elements from any of the neighbours can be moved into the unbalanced node so that both nodes (receiving and lending) become balanced (using `CanLend()` function). In this case, *redistribution* takes place (using `Redistribute()` function). However, if the *redistribution* is not possible, then the node is merged with its neighbour (using `Merge()` function). One special case that is considered by the algorithm is when the merged node is the only node left of its parent and the parent happens to be the root. In this case, the original root is removed and the merged-node becomes the new root, decreasing the height of the B-tree by one.

Algorithm 2 Algorithm for *redistribution* and *merging* of nodes on deletion

Require: *Key*

```

1: function DELETE(key)
2:   leaf ← FINDLEAF(key)
3:   REMOVEELEMENT(leaf, key)
4:   if UNDERFLOW(leaf) then
5:     DEALUNDERFLOW(leaf)
6:   end if
7: end function
8: function DEALUNDERFLOW(node)
9:   p ← GETPARENT(node)
10:  Sibling ← GETSIBLING(node)
11:  if ISEMPY(node) then
12:    REMOVEELEMENT(p, key)
13:    if UNDERFLOW(p) then
14:      DEALUNDERFLOW(p)
15:    end if
16:  else if CANLEND(Sibling) then
17:    c ← Sibling.keyCount.
18:    REDISTRIBUTE(p, node, Sibling, c - 1)
19:  else
20:    MERGE(p, node, Sibling)
21:    if p==root AND Sibling==null then
22:      FREE(p)
23:      root ← node
24:    end if
25:  end if
26: end function

```

Node-balancing plays a vital role in determining the forensic repercussions of B-trees. Due to *splitting* and *redistribution*, copies of elements are scattered across multiple leaf nodes, thereby

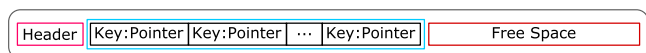


Fig. 2. Structure of an internal node showing Key–Pointer pairs.

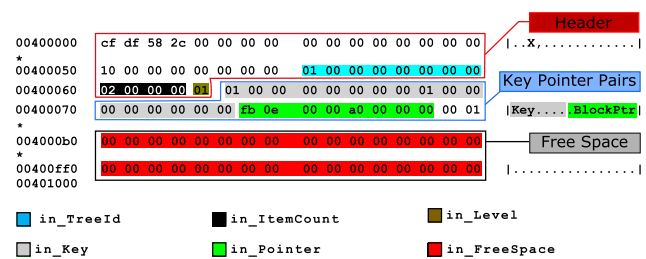


Fig. 3. Structure of an internal node of Btrfs B-tree.

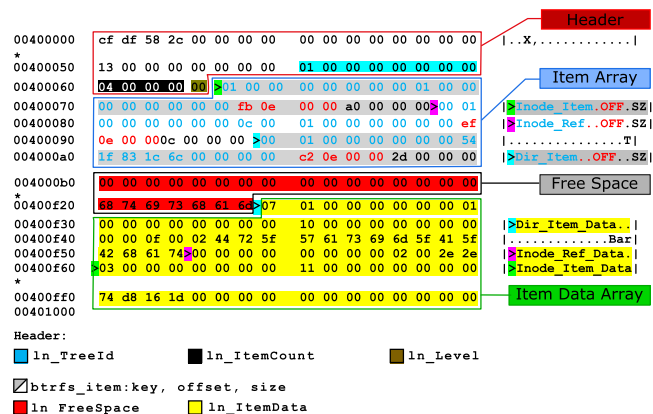


Fig. 5. Structure of a typical Btrfs generic leaf node.

increasing the chances of their recovery in case of tampering or deletion. Besides, *node-splitting* creates block slack spaces within the leaf nodes of B-tree which are bound to contain data remnants in atleast half of the cases and hence promise value. It is worth mentioning here that since Btrfs supports optimized block sizes, Btrfs B-trees do not need re-balancing frequently. However, since these nodes are not entirely full, this wastes some storage space.

Internal node

The internal nodes in Btrfs trees contain lists of [key, block-pointer] pairs, and point towards other nodes of the same tree. These nodes do not contain any information pertaining to a file or directory. Rather, they contain block pointers to other internal or leaf nodes along with their respective lowest key values as shown in Fig. 2.

This information is critical for successful traversal and parsing of file system data structures during forensic analysis. More importantly, internal nodes can provide significant forensic clues if they were previously allocated as leaf nodes. Since a major portion of the space within internal nodes remain unused, some remnants of previously allocated leaf node (now allocated as internal node) remain unaltered and hence can be retrieved. This behavior of Btrfs is because of its dependence on COW to commit any file system updates, which results into a large pool of deallocated nodes. Our experiments revealed that probability of reallocation of a previously allocated leaf node as internal node increases as the file system ages. This implies that information within internal nodes is forensically important for traversal and possible extraction of forensic evidence that exists due to COW and re-allocation from leaf nodes. Fig. 3 shows the structure of an internal node of a Btrfs B-tree.

Generic leaf node

Btrfs trees use leaf nodes to store actual information on files or directories like filenames, inode data, *extent* data and so on. A leaf node stores an *item* array in the beginning, and a reverse sorted *item-data* array at the end in such a way that the two arrays grow towards each other as shown in Fig. 4. An element of *item* array is of fixed size whereas *item-data* element is variable sized.

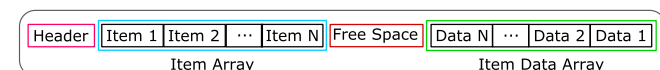


Fig. 4. Structure of a leaf node showing Items and corresponding data.

This data structure is forensically important to recover deleted files because they are the file system repositories that store both user visible content (data) and its metadata. This implies that the possibility of recovering a deleted file only exists if the node happens, or previously happened, to be a leaf node. However, in order to identify and analyse leaf nodes, the header of the node should be analysed first to verify its level, and the number of *items* that follow the header. This is because the level is used to differentiate between internal (with level 1) and leaf nodes (with level 0), which is necessary for successful traversal and possible evidence-extraction. The number of *items* determine the valid *item-count* within the node that can be used as a marker to delineate valid file system entries from invalid ones and hence is of great forensic importance.

Fig. 5 shows a typical Btrfs leaf node. The *ln_TreeId* identifies the tree to which the node belongs and the *ln_ItemCount* indicates the number of *items* that follow the node header. The *ln_Level* defines the level of the node in the tree and is always 0 for leaf nodes. Immediately following the header is the array of *btrfs_items* which point to the reverse sorted array of *ln_ItemData* at the end in such a way that the two arrays grow towards each other.

btrfs_items

The leaf nodes store information on files and directories in the form of *btrfs_items*. The *btrfs_item* is a generic data structure which is composed of *btrfs_disk_key*, *itm_DataOffset*, and *itm_DataSize* that identifies an object and its associated data. These *btrfs_items* identify directory and file names, directory indexes, file content, MAC-DTS, etc. as are identified by NTFS *attributes* like \$FILE_NAME, \$INDEX_ROOT, \$DATA, and \$STANDARD_INFORMATION on an NTFS volume (Nagar, 1997). A generic *btrfs_item* is shown in Fig. 6.

The *btrfs_disk_key* contains the *object-id* of the file or directory, and is associated with each and every *btrfs_item*. The *btrfs_disk_key* is composed of *itm_ObjectId*, *itm_Type*, and *itm_KeyOffset* as shown in Fig. 7.

Since a leaf node contains *items* of various types belonging to various objects, the *itm_ObjectId* relates different *item-types* to their respective object. As *itm_ObjectId* makes up the most significant bits of the *key*, *key* ordering causes all of the *items* to be

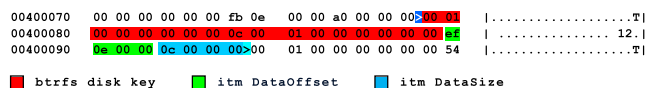


Fig. 6. Structure of a generic btrfs_item.

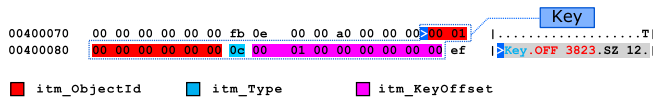


Fig. 7. Components of `btrfs_disk_key`.

logically grouped together in a B-tree. Hence, all the data associated with a given `itm_ObjectId` is packed together either in one block or spans over neighbouring blocks. This makes `btrfs_items` fundamental to deleted-file recovery. In addition, an object typically comprises of several *items*. To differentiate between multiple *items* associated with a given `itm_ObjectId`, Btrfs uses `itm_Type` field of the `btrfs_disk_key`. Fig. 8 lists all the `itm_Type` types associated with `btrfs_items` in the Btrfs file system.

The `btrfs_dir_item` stores directory entries (files or directories) of the parent directory (identified by *object-id*) along with their names. The `btrfs_inode_item` contains the most valuable information from a forensic point of view, i.e., MAC-DTS. Btrfs supports nanosecond resolution for three timestamp fields of the *stat* structure. Nanosecond resolution was introduced with ZFS, and starting with Linux kernel 2.5.48, the *stat* structure supports nanosecond resolution for the three file timestamp fields, which are exposed as additional fields in the *stat* structure (Wikipedia, 2018). Nanosecond resolution improves accuracy which is central to timeline reconstruction of events in a forensic investigation. The `Btrfs_file_extent_item` contains or references actual file content depending upon the file size. Smaller files are stored in *inline-extents* within the leaf whereas larger files are stored outside the file-system tree in contiguous blocks as *regular-extents*. Conceptually, the idea is similar to *resident* and *non-resident* \$DATA attribute of NTFS (Bhat and Quadri, 2011). Both *extent* types are defined by the `btrfs_file_extent_item`. However, the amount of metadata stored is more in case of *regular-extents*.

Since `btrfs_items` form the building blocks of all data stored on the file system, they are fundamental in identification and extraction of forensic evidence that exist due to *redistribution* and *merging* of nodes.

Inode core

The inodes are stored in `btrfs_item` of type `btrfs_inode_item` at offset zero in the *key*, and have a type value of 1. Inode *items* are always the lowest valued *keys* for a given object, and store the traditional *stat* data for files and directories. Unlike other file systems, the inode structure is relatively small, and does not contain embedded file data, *attribute* data, or *extent* mappings. They are stored in other *item* types. The most valuable information that an inode contains from a forensic point of view is MAC-DTS (Bhat and Quadri, 2013). The timestamp field is defined as a `timespec`

Item Type	Hexadecimal Value	Item Type	Hexadecimal Value
INODE_ITEM	01	ROOT_REF	9C
INODE_REF	0C	EXTENT_ITEM	A8
INODE_EXTREF	0D	TREE_BLOCK_REF	B0
XATTR_ITEM	18	EXTENT_DATA_REF	B2
ORPHAN_ITEM	30	EXTENT_REF_V0	B4
DIR_LOG_ITEM	3C	SHARED_BLOCK_REF	B6
DIR_LOG_INDEX	48	SHARED_DATA_REF	B8
DIR_ITEM	54	BLOCK_GROUP_ITEM	C0
DIR_INDEX	60	DEV_EXTENT	CC
EXTENT_DATA	6C	DEV_ITEM	D8
EXTENT_CSUM	80	CHUNK_ITEM	E4
ROOT_ITEM	84	STRING_ITEM	FD
ROOT_BACKREF	90		

Fig. 8. btrfs_item types and their respective Hexadecimal values.

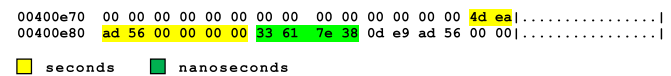


Fig. 9. Second & nanosecond resolution employed by Btrfs timestamp

structure, which represents the time in seconds and nanoseconds components as shown in Fig. 9.

The components of Btrfs timestamp structure are shown in Fig. 10.

The `st_atime` field represents access time and is changed after every file access. The `st_ctime` denotes modification time and is changed by writing or setting inode information (i.e., owner, group, link count, mode, etc.). The field `st_mtime` is changed by file modifications, for example, by `truncate`, `utime` and `write` operation. Moreover, `st_mtime` of a directory is changed by the creation or deletion of files in that directory. The `st_mtime` field is not changed for changes in owner group, hard link count, or mode. The field `st_otime` is reserved for future use.

All this information is forensically important in timeline reconstruction of various states of file system data, and the nano-second resolution improves the accuracy of forensic investigation.

Directory

Btrfs expresses a directory or its entries with *items* of type `btrfs_dir_item` and `btrfs_dir_index`. `btrfs_dir_item` is a structure comprising of filename, type and `btrfs_disk_key` specifying the location of the object whereas `btrfs_dir_index` is a copy of `btrfs_dir_item` sorted by inode sequence number. Basically, it follows the structure of leaf node and `btrfs_item` to define and identify its contents. The `btrfs_dir_item` and `btrfs_dir_index` are conceptually similar to `$FILE_NAME` and `$INDEX_ROOT` attributes of NTFS, respectively, since they also describe directory metadata in NTFS (Nagar, 1997). From a forensic perspective, the elements of interest in `btrfs_dir_item` and `btrfs_dir_index` are shown in Fig. 11.

The `d_Key` gives the location of the directory object in file-system tree and the `d_Type` differentiates whether the entry is a directory, file or a block device. The actual content is expressed with `d_NameLen`, `d_Name[NameLen]` and the `itm_ObjectId` value of the `d_Key`. This information is forensically important in determining how the file system organises data in terms of directory structure. Thus, it is important to understand their structure for successful identification and reconstruction of directory structure during forensic investigation.

Regular file

The B-tree file system represents a regular file using the `btrfs_file_extent_item` and B-tree structure. The `btrfs_file_extent_item` records a generation number, its size, compression, encryption, and type. The generation number records when the *extent* was created and represents the serial number of the latest checkpoint that updated it. They are used by the file system to identify phantom records and the latest updated copy of an *extent* in case of multiple snapshots (Rodeh et al., 2013). The file B-tree structure is similar to the directory B-tree structure described above.

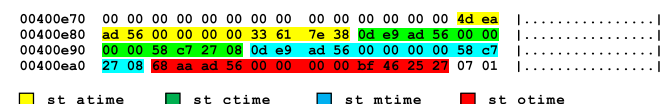


Fig. 10. Structure of Btrfs timestamp data structure.

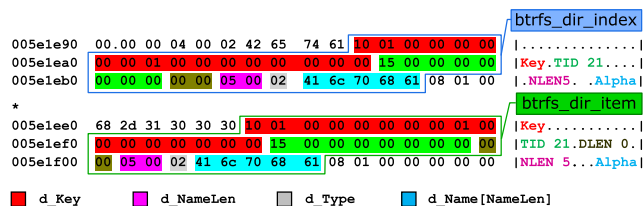


Fig. 11. Components of btrfs_dir_item and btrfs_dir_index.

An *extent* is a basic structure that identifies a file in the B-tree file system. Small files that occupy less than one leaf block are packed into the B-tree inside an *inline-extent* which is similar to *resident \$DATA* attribute in NTFS (Bhat and Quadri, 2011). In this case, the *key offset* is the byte offset of the data in the file, and the *size field* of the *item* indicates how much data is stored. There may be more than one of these per file. Larger files are stored using *regular-extents* which is similar to *non-resident \$DATA* attribute in NTFS (Bhat and Quadri, 2011). These are contiguous on-disk areas that hold user data without additional headers or formatting. For *regular-extents*, the *btrfs_file_extent_item* records an additional [disk block, length] pair to record the area on disk, and a [logical file offset, length] pair to record the file area. Basically, it is a mapping from a logical area in a file to a physical area on disk. The *btrfs_file_extent_item* follows the structure of *btrfs_item* within the leaf node to define and identify its contents. Fig. 12 shows the structure of a typical *btrfs_file_extent_item*.

The field *ext_NBytes* is the size of the decoded *extent*. The *ext_Type* field differentiates between *inline*, *regular* and *pre-alloc extents* as shown in Fig. 13. If the *extent* is *inline*, the remaining bytes are the data bytes (*ext_NBytes* in case no compression/encryption/other encoding is used). Otherwise the structure continues with *ext_Address*, *ext_Size*, *ext_Offset* and *ext_LBytes*. The *ext_Address* is the logical address of the *extent*. If this is zero, the *extent* is sparse and consists of all zeroes. *ext_Size* is the size of the *extent* and *ext_LBytes* denote the logical number of bytes in file. This information is important in identifying forensic evidence stored within file extents.

Node-balancing and evidence-extraction

Node-balancing is central to evidence-extraction in Btrfs. Due to *redistribution* and *merging*, copies of both metadata and data are scattered across multiple leaf nodes. This makes it possible to recover the previous state of a file or directory in case of tampering or deletion, by analyzing the *items* that exist beyond the *item-count* value of the leaf. These *items* are called *Orphan-Items*, as they are not part of file system structure and are deemed invalid by Btrfs.

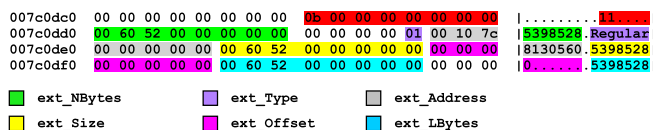


Fig. 12. Structure of btrfs_file_extent_item.

Extent Type	Hexadecimal Value
Inline Extent	00
Regular Extent	01
Pre-alloc Extent	02

Fig. 13. btrfs_file_extent_item types and their respective Hexadecimal values.

Whether we consider deletion of a single file in a directory, or the directory itself, the result has no effect on data recovery. This is because Btrfs uses B-trees wherein *items* and *item-data* of an object can span over multiple neighbouring nodes. When a file or an entire directory is deleted, the number of *items* in a leaf node change, and the node is COWed to another location. This means that both data and metadata pertaining to the deleted content still persist at the previous location in the form of *items* and *item-data* respectively, and can be recovered. However, whether the new node contains the information on the deleted content or not, depends upon some factors explained in the following sections.

Directory deletion operation

Recovery of metadata pertaining to directories and directory entries from newly COWed nodes is governed by *merging* and *redistribution* of nodes, and varies as follows:

Merging

If data deletion at tree level occurs in such a way that it results in *merging* of nodes, most of the data pertaining to the deleted content is overwritten in the new node by *items* from neighbouring node as a consequence of *merging*. In this case, the newly COWed node may contain only some fragments of deleted data within *Orphan-Items* as shown in Fig. 15, given the degree of *merging* is low, and hence only the old node remains to be a viable source for data recovery.

To demonstrate *merging*, six directories *Delta*, *Echo*, *Houston*, *Jake*, *Kali*, and *Lima* were created on the file system. Fig. 14 shows the directory entries before deletion operation was performed.

When the directory entries of *Lima*, *Kali*, *Jake* and *Houston* were

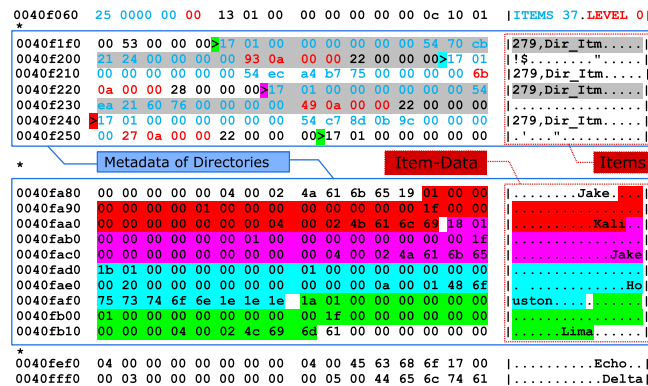


Fig. 14. Directory entries before deletion.

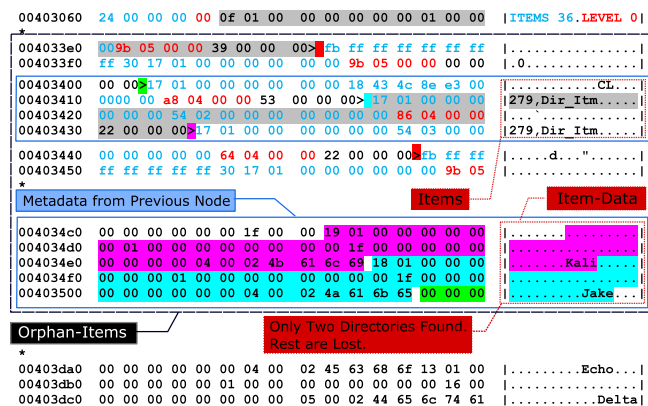


Fig. 15. Deleted directory data after deletion (Nodes merged).

deleted, the node was COWed to another location. On examining the newly COWed node, it was found that most of the data pertaining to deleted content was lost as shown in Fig. 15.

Furthermore, analysis of file-system tree structure revealed that the loss of data due to deletion operation was a direct consequence of *merging* of nodes. Fig. 16 shows the file-system tree layout before and after the deletion operation. It is clear from the figure that deletion operation reduced the number of *items* to a value small enough to unbalance the tree that resulted in *merging* of nodes.

Redistribution

On the contrary, if the deletion results in *redistribution* of *items*, the `btrfs_dir_items` referring to the deleted content still exist in the new node as *Orphan-Items* as shown in Fig. 18. However, the amount of data that can be salvaged depends upon the degree of *redistribution* which is low if fewer records are relocated from adjacent nodes. This implies that although information on deleted content is persistent in the old node, it can be harvested from the new node as well.

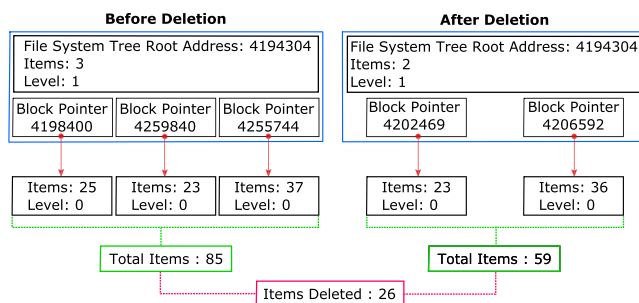


Fig. 16. Logical layout of file system B-tree before and after deletion (Nodes merged).

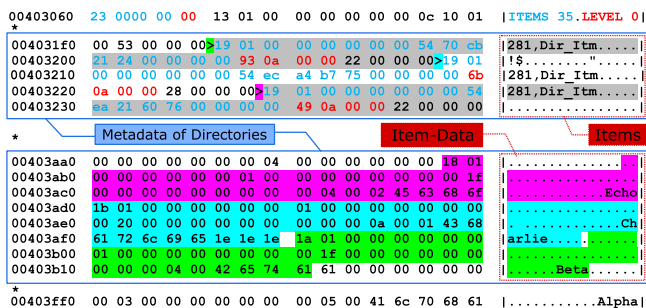


Fig. 17. Directory entries before deletion.

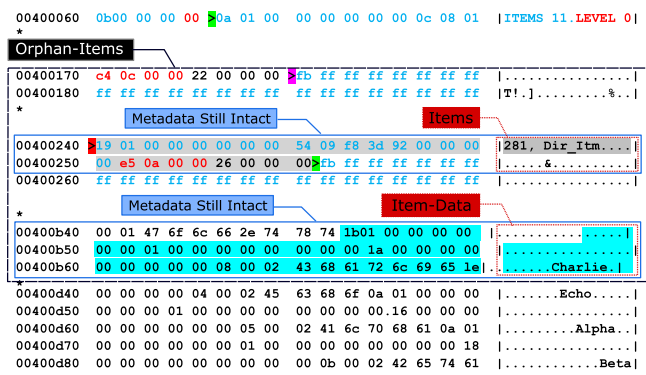


Fig. 18. Deleted directory-data after deletion (Nodes redistributed).

To test *redistribution*, sample directories *Beta*, *Charlie* and *Echo* were created on the file system. Fig. 17 shows the metadata of directories along with other *items* already present on the file system before the deletion operation was performed.

The deletion of directory *Charlie* resulted in a node being COWed to another location. When the COWed node was examined for traces of deleted data, it was found that the deleted content was still available in COWed node beyond `ln_ItemCount` value in the form of *Orphan-Items* as shown in Fig. 18.

This implies that it is possible to move to the *items* of deleted directory, and further, to recover the deleted directory and build the directory structure. However, thorough analysis of file system B-tree revealed the fact that it was *redistribution* that was responsible for presence of deleted content in the COWed node. It is clear from Fig. 19 that the deletion of directory *Charlie* changed the number of *items* in such a way that it forced to redistribute *items* from neighbouring nodes in order to balance the B-tree. This can also be attributed to the fact that `btrfs_dir_items` are relatively very small in size, and hence their deletion mostly results in *redistribution* particularly on an aged file system.

It is safe to argue that although deleted data and metadata is persistent as a consequence of COW technique, it can be recovered from the newly spawned nodes as well. However, the amount of data that can be recovered in latter case is governed by the degree of *merging* or *redistribution* of nodes.

Regular-file deletion operation

Regular file recovery is driven by the recovery of data stored in file *extents*. However, whether file data is stored within *regular-extent* or in *inline-extent*, the result has significant impact on the overall file recovery.

In case a file is stored in a *regular-extent* outside the leaf block, the `btrfs_file_extent_item` defining the file data is treated just like any other *item*. This is because these items do not store file content but reference the location where the actual file data is stored as shown in Fig. 20. Hence, the *items* and *item-data* defining the file, which is stored in *regular-extent* exhibit similar behavior as that of directory. This implies that file deletion mechanism of a file stored in a *regular-extent* is similar to that of a directory. Therefore, it can be safely argued that the recovery mechanism of a file stored in a *regular-extent* is similar to that of directory because all the data required to identify the *regular-extents* is resident within the leaf node.

On the contrary, *inline-extents* store file content within the leaf block along with other metadata information via `btrfs_file_extent_items`, making them one of the largest `btrfs_items` in terms of size. Although, *inline-extents* also follow the same recovery mechanism as that of directory, the size of the file stored within *inline-extents* is significant in determining the node-

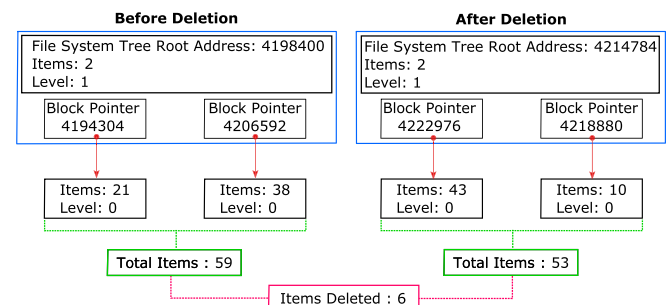


Fig. 19. Logical layout of file system B-tree before and after deletion (Nodes redistributed).

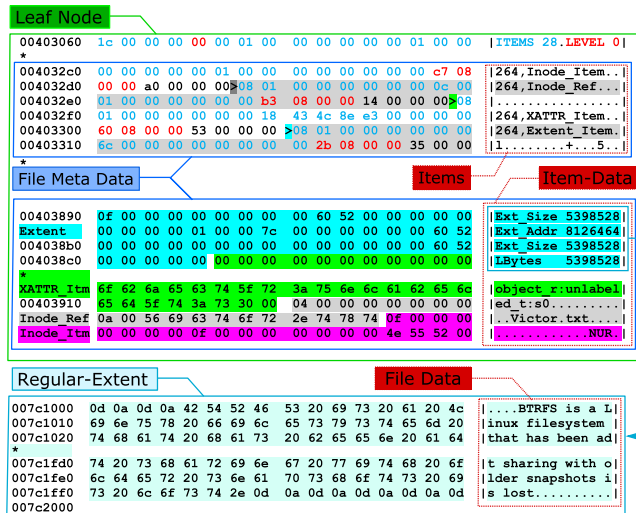


Fig. 20. Metadata of a file stored in a regular-extent.

balancing mechanism of the file system on deletion. This in turn affects the final file recovery as discussed in the following sections.

Merging

If the size of file deleted at tree level is more than half of block size, the memory allocation of the leaf is reduced significantly, and merging of nodes is guaranteed. As such, most of the data, particularly file extents (file content) pertaining to the deleted content is overwritten in the new node as a consequence of merging. Besides, the degree of merging is much worse which can be directly attributed to block slack space created on deletion. As a result, most of the data is lost completely as a consequence of merging and can not be recovered from the newly COWed node. In such cases, only the old node remains to be a viable source for data recovery.

To demonstrate merging for inline-extents, sample file *Con. txt* of 2.1 KB size was created on the file system. Fig. 21 shows the file extent and other metadata of *Con. txt*.

When the regular file *Con. txt* was deleted, most of the data pertaining to the file was lost as shown in Fig. 22.

Further analysis of file-system tree structure revealed that the loss of data due to deletion can be directly attributed to merging of nodes. Fig. 23 shows the file-system tree layout before and after the

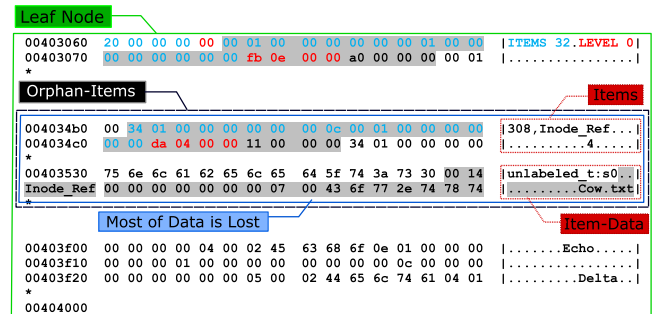


Fig. 22. Deleted file data after deletion (Nodes merged).

deletion operation. It is clear from Fig. 23 that the degree of merging was worse, as the number of items in the resulting leaf node increased significantly, thereby overwriting any available forensic evidence. Also, given the size of deleted data, full recovery was far from possible.

Redistribution

On the other hand, if the file size is relatively small, redistribution of items is guaranteed in most of the cases. This implies that the *btrfs_inode_ref* and the *btrfs_file_extent_items* referring to filename and file content (respectively) still exist in the new node in the form of *Orphan-Items*. Furthermore, such file sizes also cause relocation of fewer records from neighbouring nodes resulting in low degree of redistribution. As a result, the amount of data that can be recovered from the newly COWed node is sufficient in recovering the deleted file. This implies that the deleted content can be successfully reconstructed from the new node without much effort.

To test redistribution in case of small files, sample file *Jake. txt* of size 36-bytes (containing a text string) was created on the file system. Fig. 24 shows the file's inline-extent and filename metadata along with other items present on the file system before the deletion operation was performed.

The deletion of file *Jake. txt* results in a node being COWed to another location. When the COWed node was examined for traces of deleted data, it was found that the deleted content was still available in the node beyond *ln_ItemCount* value as *Orphan-Items* as shown in Fig. 25.

This implies that although redistribution was responsible for presence of deleted content in the COWed node, it was the small size of *Jake. txt* that led to redistribution in the first place. It is clear from Fig. 26 that the deletion of regular file *Jake. txt* changed the number of items in such a way that it forced the redistribution of items from neighbouring nodes in order to balance the B-tree. It is worth mentioning here that in case of regular files stored in inline-extents, redistribution occurs more frequently and can be primarily

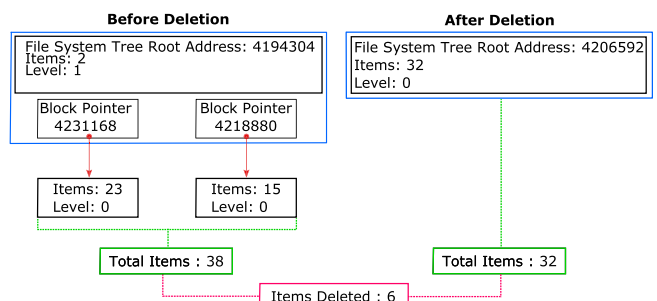


Fig. 23. Logical layout of file system B-tree before and after deletion (Nodes merged).

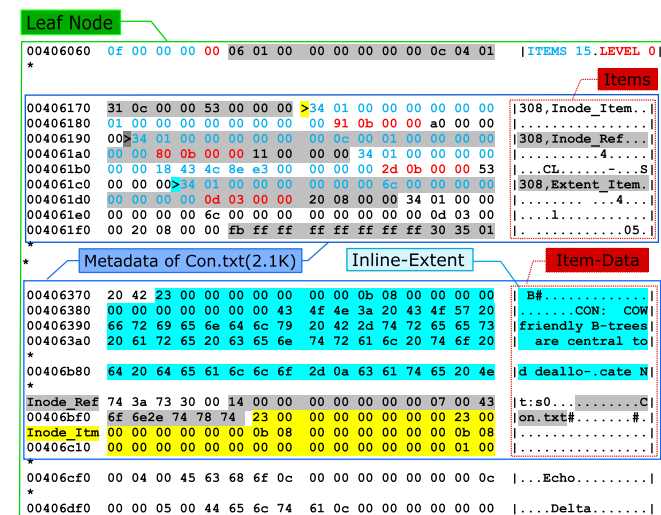


Fig. 21. Filename and file extent entries before deletion.

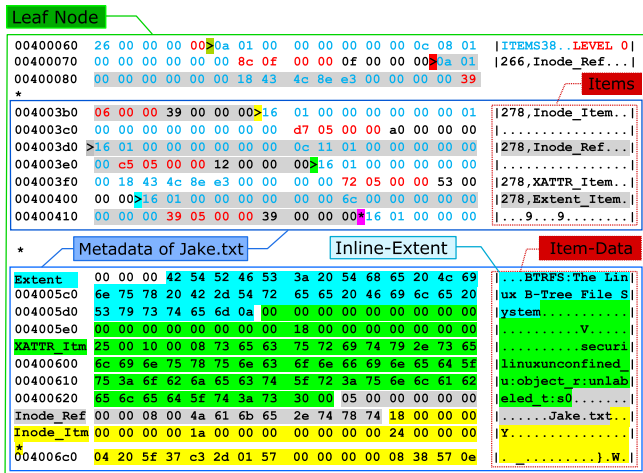


Fig. 24. Filename and file-extent entries before deletion.

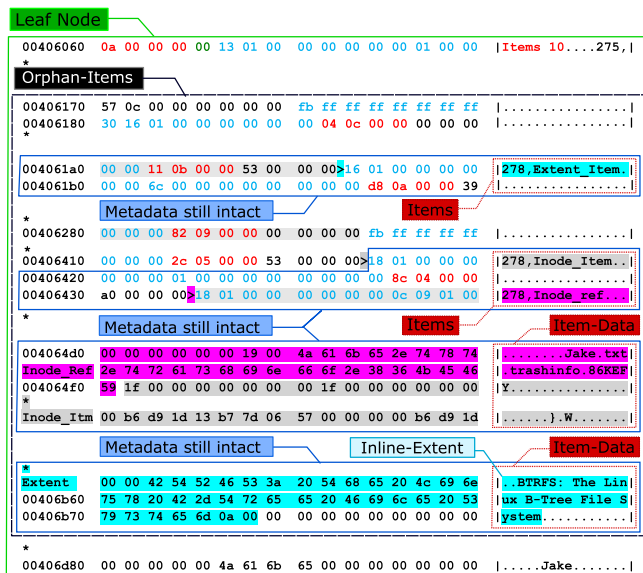


Fig. 25. Deleted file data after deletion (Nodes redistributed).

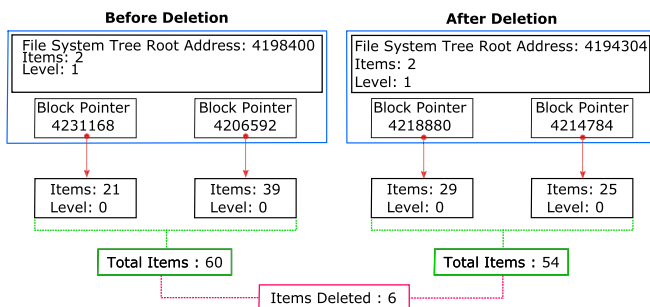


Fig. 26. Logical layout of file system B-tree before and after deletion (Nodes redistributed).

attributed to deletion of small sized files.

From this we conclude that although recovery techniques are almost same for both *extent* types, the chances of reconstructing the original deleted-file are higher for files stored in *regular-extents* and lower for files stored in *inline-extents*. In addition, the

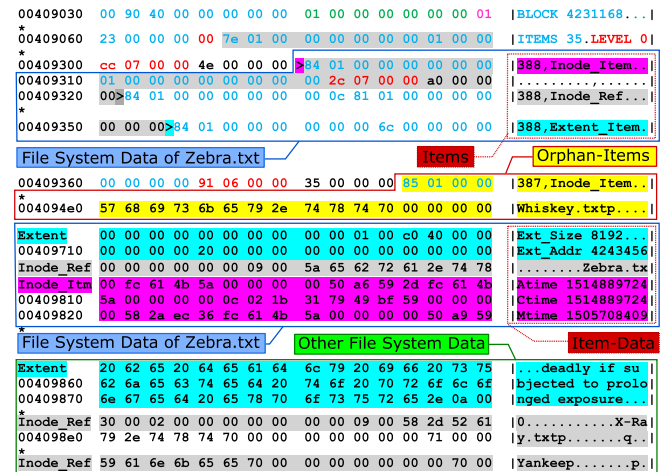


Fig. 27. Metadata of a file before renaming and timestamp modification.

information in the old block is still available and the actual file data is not overwritten when the file is deleted.

Other operations on file & directories

Besides analysing the impact of node-balancing on Btrfs data structures during deletion operation, we also investigated them for other file system operations like rename, move, modifying contents and MAC-DTS, and so on.

Modifying MAC-DTS and filename

Modifying MAC-DTS and file or directory names neither results in *merging* nor *redistribution*. This is because these metadata operations do not alter the memory constraints of leaf nodes necessary to cause reshuffling of *btrfs_items*.

In order to test the effect of timestamp modification and rename operation, sample file *Zebra.txt* was created on the file system. Fig. 27 shows the file *extent* of *Zebra.txt* along with other *items* including *Orphan-Items* already present on the file system before rename and timestamp modification. As *Zebra.txt* was renamed to *Zebbbbra.txt*, the node was COWed to another location. However, due to COW updates, the rename operations also changed the MAC-DTS of *Zebra.txt* as shown in Fig. 28. On close examination and comparative analysis, it is clear from Figs. 27 and 28 that the rest of file system data (including *Orphan-Items*) was retained in the COWed node. Interestingly, the *btrfs_items* including *Orphan-Items* in the COWed node follow the same order as that of old node. Even the *ln_ItemCount* value that defines the number of valid *btrfs_items* in the node did not change, and remained the same. This implies that although changes in filename and timestamps are reflected in the COWed node as a result of rename operation, the modifications are strictly localized to their respective data structures without causing any shuffle among other *items* of the node. From this, we infer that neither *merging* nor *redistribution* has happened, and the chances of recovering the previous name or MAC-DTS of the file (or directory) from the newly COWed node are significantly less. Hence, only the older nodes remain to be the viable source of retrieving any forensic data in such cases.

Modifying file contents and location

Modifying contents of a file, or moving files or directories, can significantly change file system data structures or their ordering, which in turn may result in *merging* or *redistribution*. Depending upon the number of *items* in a leaf node, the type of file *extent* and

00403030	00 30 40 00 00 00 00 00	01 00 00 00 00 00 00 01	BLOCK 4206592...
00403060	23 00 00 00 00 7e 01 00	00 00 00 00 00 01 00 00	ITEMS 35.LEVEL 0
00403300	c8 07 00 00 4e 00 00 00	84 01 00 00 00 00 00 00	388,Inode_Item...
00403310	01 00 00 00 00 00 00 00	28 07 00 00 a0 00 00 00	388,Inode_Ref...
00403320	00 84 01 00 00 00 00 00	0c 81 01 00 00 00 00 00	388,Extent_Item...
00403350	00 00 00 84 01 00 00 00	00 00 00 6c 00 00 00 00	
File System Data of Zebbra.txt			
00403360	00 00 00 00 8b 06 00 00	35 00 00 00 85 01 00 00	387,Inode_Item...
004034e0	57 68 69 73 6b 65 79 2e	74 78 74 70 00 00 00 00	Whiskey.txt...
Extent			
00403710	00 00 00 00 01 00 c0 40	00 00 00 00 00 00 20 00	Ext Size 8192...
00403710	00 00 00 00 00 00 00 00	00 00 00 00 00 00 20 00	Ext Addr 4243456
Inode_Ref	0b 00 5a 65 62 62 62 72	61 2e 7478 74 70 00 00	Zebbra.txt...
Inode_Item	00 00 00 00 00 00 00 00	00 00 00 00 00 fc 61 4b	Atime 1514889564
00403800	5a 00 00 00 00 50 a6 59	2d 56 69 4b 5a 00 00 00	Ctime 1514891606
00403810	00 5c 68 c8 31 79 49 bf	59 00 00 00 00 58 2a ec	Mtime 835217500
File System Data of Zebbra.txt			
Extent	64 65 61 64 6c 79 20 69	66 20 73 75 62 6a 65 63	deadly if subje
00403860	74 65 64 20 74 6f 20 70	72 6f 6c 6f 6e 67 65 64	ted to prolonged
00403870	20 65 78 70 6f 73 75 72	65 2e 0a 00 00 00 00 00	exposure...
Inode_Ref	00 00 00 00 00 09 00	58 2d 52 61 79 2e 74 78	X-Ray.txt...
Inode_Ref	30 00 02 00 00 00 00 00	00 00 06 00 59 61 6e 6b	Yank...
00403a30	65 65 70 00 00 00 00 00	00 00 72 00 00 00 00 00	eep...x...

Fig. 28. Metadata of a file after renaming and timestamp modification.

whether the file size increases or decreases after modifying its contents, *merging* or *redistribution* may occur. In such cases, procedure for retrieving previous data or state is same as that of regular files, and is described in section 4.2. On the other hand, in case of a move operation, the chances of *merging* are least but *redistribution* is bound to occur. This is due to the fact that move operation simply changes the file hierarchy without generating any additional content. Moreover, as Btrfs forces adjacent storage of multiple *items* belonging to a particular file or directory based on their *object-id*, a move operation may necessitate *redistribution* due to change in parent *object-id* value of the moved file. This implies that the previous location of the file or directory can be easily retrieved by analysing the *Orphan-Items* in newly COWed nodes as discussed earlier.

In order to demonstrate *redistribution* in case of move operation, sample file *X-Ray.txt* was created on the file system as shown in Fig. 29. It is clear from Fig. 30 that *X-Ray.txt* had an *object-id* of 387 with parent *object-id* of 386. As the sample file *X-*

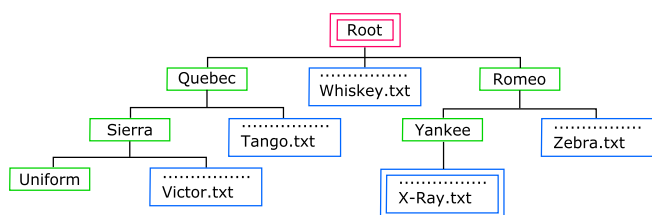


Fig. 29. Directory structure for testing move operation.

00409060	23 00 00 00 00 17 01 00	00 00 00 00 00 01 00 00	ITEMS 35.LEVEL 0
00409270	00 00 82 01 00 00 00 00	00 00 54 93 cc 35 3e 00	386-Dir_Item...
004092f0	01 00 00 00 00 00 00 00	6c 00 00 00 00 00 00 00	387-Extent_Item...
File System Data of X-Ray.txt			
004093b0	05 00 00 53 00 00 00 00	85 01 00 00 00 00 00 6c	389-Extent_Item...
004095d0	00 00 30 00 00 00 00 00	00 00 00 00 00 01 00 e0	Ext Addr 4251648
004095d0	40 00 00 00 00 00 00 00	30 00 00 00 00 00 00 00	Ext Size 12288...
387-Extent	20 62 65 20 64 65 61 64	6c 79 20 69 66 20 73 75	be deadly if su
00409860	62 6a 65 63 74 65 64 20	74 6f 20 70 72 6f 6c 6f	bjected to prolo
004099a0	09 00 01 58 2d 52 61 79	2e 74 78 74 83 01 00 00	Key 387.....
386-Dir	00 00 00 00 00 00 09 00	00 01 58 2d 52 61 79 2e	X-Ray...
File System Data of X-Ray.txt			
385-Dir	65 84 01 00 00 00 00 00	00 01 00 00 00 00 00 00	Key 388.....
385-Dir	65 62 72 61 2e 74 78 74	82 01 00 00 00 00 00 00	ebra.txt.Key 386
00409b60	00 00 06 00 02 59 61	6e 6b 65 65 00 00 00 00	Yankee....

Fig. 30. Metadata of a file before move operation.

Ray.txt was moved from its parent directory(386-Yankee) to its root directory(379-Root in this case), all the nodes affected by the operation were COWed to new locations. Fig. 31 shows how the move operation caused a change in the metadata of *X-Ray.txt*. The *object-id* of the file's parent has changed to 379(Root) while its *object-id* has remained the same, thereby disrupting the logical ordering of file system B-tree, and resulting in *redistribution* of nodes.

Furthermore, it can be argued that Btrfs's native support for snapshots and RAID arrays (Powell, 2012) along with COW transactional model ensures availability of data and metadata both in allocated and unallocated space. Snapshots include both metadata and content (data) representing a treasure trove of temporal and state information. On the other hand, Btrfs's RAID functionality guarantees metadata duplication even on a single device. This implies that in addition to the multiple copies of content and metadata that exists in unallocated space due to the COW transactional model, multiple allocated copies of metadata and/or content will also be available via Btrfs's RAID and snapshot functionality.

Evidence-extraction procedure

The main aim of all digital forensic investigations is identification and extraction of digital evidence as it gives valuable insights into the events that might have transpired leading to the matter under investigation. As file systems are getting increasingly more sophisticated with inclusion of complex compression and encryption techniques, and exponential increase in storage sizes, use of *brute-force* and *semantic-carving* techniques for evidence-extraction is becoming impractical. As such, time-sensitivity and resource constraints of forensic investigations call for efficient extraction of forensic evidence from devices.

Based on the identified forensically important data structures of Btrfs, the observed changes that they incur due to node-balancing during file system operations and the forensic evidence that they create in the form of *Orphan-Items*, we propose a 6-stage evidence-extraction procedure as shown in Fig. 32. The proposed technique ensures efficient extraction of forensic evidence by not only carving data from leaf nodes but from internal nodes as well. Since Btrfs uses same data structures for both metadata and content, there is no clear distinction between the two. Hence, the procedure for the extraction of metadata and content as potential evidence sources remains the same. The procedure also facilitates reconstruction of entire directory structure by analysing various file system data structures recovered during the extraction process. Furthermore, the extraction of additional information, such as *extent* names, generation numbers, and MAC-DTS timestamps (which are critical for event-reconstruction) is also ensured.

00409060	1c 00 00 00 00 17 01 00	00 00 00 00 00 01 00 00	ITEMS 28.LEVEL 0
00409110	24 00 00 00 00 7b 01 00	00 00 00 00 54 84 a2 84	379-Dir_Item...
00409120	1e 00 00 00 00 50 0d 00	00 23 00 00 00 75 01 00	379-Dir_Item...
File System Data of X-Ray.txt			
00409440	00 00 00 00 00 f0 05 00	00 28 00 00 00 82 01 00	386-Dir_Item...
386-Dir	00 70 00 00 00 00 00 00	00 00 00 06 00 02 58 2d	Key 387.....X-
00409ba0	52 61 79 2e 80 01 00 00	00 00 00 00 01 00 00 00	Ray.....
380-Dir	00 01 54 61 6e 67 6f 2e	74 78 74 00 00 00 00 00	Key 384.Tango.txt
379-Dir	75 65 62 65 63 85 01 00	00 00 00 00 00 01 00 00	Key 389.Whiskey.txt
Other File System Data			
379-Dir	00 00 00 00 00 00 00 00	00 09 00 01 58 2d 52 61	Key 387.....X-Ra
379-Dir	79 2e 74 78 74 81 01 00	00 00 00 00 00 01 00 00	y.txt.....Key 385
File System Data of X-Ray.txt			
Item-Data			

Fig. 31. Metadata of a file after move operation.

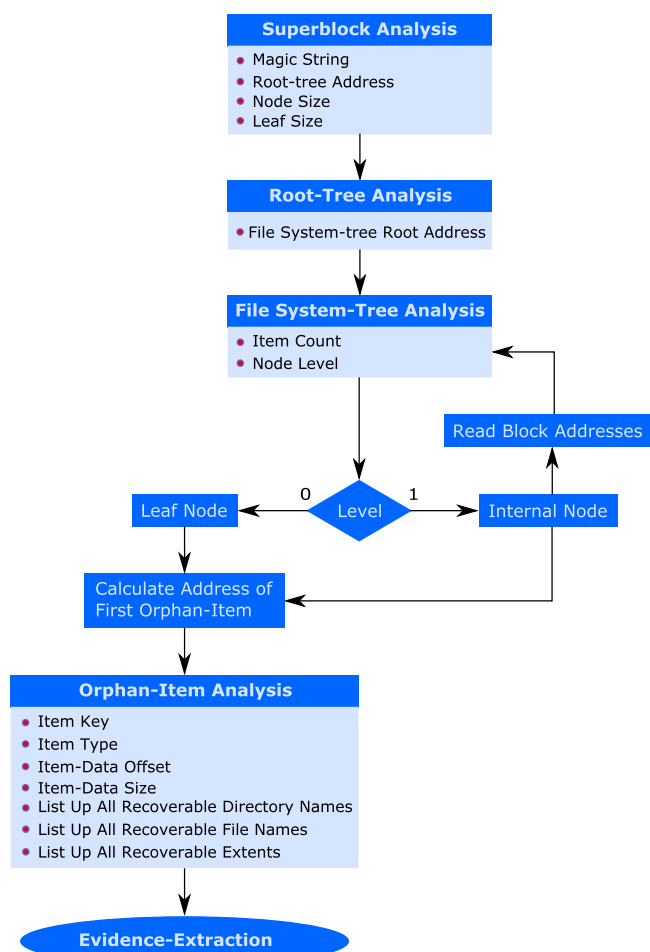


Fig. 32. Proposed evidence -extraction procedure for Btrfs

Superblock analysis stage

In Superblock Analysis (SA) stage, all the necessary information required for validation, parsing and interpretation of file system structures is extracted by analysing the Superblock. In particular, one of the most important pieces of information that Superblock holds is the node size, leaf size and the *root-tree* address. These three data structures form the basis for the entire file system to be parsed.

Root-tree analysis stage

In the Root-Tree Analysis (RA) stage, the *root-tree* located at `sb_RootTreeAddr` is analysed for root items which contain logical addresses of all the B-trees making up the file system including the *file-system tree*. The *file-system tree* address can be obtained from the *root-tree* by parsing all the root items and extracting all the logical addresses. The logical address of *file-system tree* is the one with an `inode_ref` containing the value "default".

File system-tree analysis stage

In the File System-Tree Analysis (FA) stage, the tree nodes are analysed for `ln_ItemCount` and `ln_Level` contained in the header. Both these data structures are critical for calculating the effective address that is used by subsequent stages. The information obtained from `ln_ItemCount` is used to determine the starting

address of the first *Orphan-Item* within the node whereas `ln_Level` helps in distinguishing between internal and leaf nodes. The internal nodes contain *key-pointer* pairs which point to other nodes in the *file-system tree*. These pointers or block addresses are extracted from internal nodes and passed back to File System-Tree Analysis stage for recursive analysis. Furthermore, once all the *key-pointer* pairs contained within the node are processed, the logical address of the first *Orphan-Item* is calculated. On the other hand, in case of leaf nodes, the starting address of first *Orphan-Item* is calculated by moving past all items defined by `ln_ItemCount`.

Orphan-item analysis stage

The Orphan-Item Analysis (OA) stage is concerned with extraction and analysis of *Orphan-Items*. This stage performs a sequential analysis of all the nodes (both internal and leaf nodes) for *Orphan-Items* using the address calculated in FA stage. Every *Orphan-Item* is extracted and analysed for `itm_ObjectId`, `itm_Type`, `itm_DataOffset` and `itm_DataSize` which are fundamental for identification and retrieval of its data. The `itm_Type` of an *Orphan-Item* is compared with all known *item-types* of Btrfs and only those *item-types* are retained that are associated with directories and files. In addition, when an *item* suspected of being deleted is found, its `itm_DataOffset` and `itm_DataSize` values are compared with `sb_LeafSize` in order to filter out those *items* that may contain garbage values. Furthermore, the `itm_ObjectId` within the `btrfs_disk_key` ensures that multiple *item-types* belonging to a particular file or directory are grouped together, thereby allowing for accurate reconstruction of actual directory structure during final extraction. This enables investigators to gather further information, which can be of significant value from the forensic perspective such as storage routes, deletion patterns, file names, and so on.

Evidence-extraction stage

The last stage is the actual extraction of forensic evidence. In this stage deleted files and directories are extracted using the information gathered in the previous stage, such as name, directory data, and *extent* data. The recovered *extent's* data be it *regular* or *inline* is saved in files and named as per their respective `inode_ref` value whereas directories and directory entries are saved in a way to reconstruct the original directory structure as closely as possible.

Findings

Evidence extraction in fresh Btrfs is difficult

Generally in conventional file systems, a file deleted in a newly created file system has a higher degree of recoverability. This is because they make use of in-place editing to invalidate the file (Bhat and Quadri, 2012a). This leaves metadata remnants of the deleted file as part of the file system structure. Since the deleted-file data is still present on the volume and in a fresh file system there are little chances that this data may be overwritten by subsequent file system operations, recovery can be achieved easily with high probability (Bhat and Quadri, 2012b). However, the same can not be said about Btrfs. This is primarily due to COW updates and existence of *Orphan-Items*. Due to COW, the out-of-place editing happens when a file is deleted and the newly created node does not retain metadata pertaining to the deleted file, while as the old node that contains the deleted-file metadata is not part of file system structure. Though the deleted-file data is still present on the volume, yet COW makes conventional procedures of file recovery on a fresh Btrfs volume impractical.

However, existence of *Orphan-Items* within newly created leaf nodes, which are the result of *redistribution* and *merging*, can be exploited for recovery of the deleted file. These *Orphan-Items* exist within the leaf nodes once we move past valid `btrfs_items` defined by `ln_ItemCount` value, signifying a previously deleted file entry. But for these *Orphan-Items* to exist, the file system needs to age. This is because the possibility of recovering a deleted file from *Orphan-Items* requires that these *Orphan-Items* be located towards the core of the node. Btrfs stores files as `[item, item-data]` pairs with an array of *items* at the beginning, and a reverse sorted *item-data* array at the end in such a way that the two arrays grow towards each other. As a result, *items* that are closer to the start or end of leaf node are most likely to be overwritten during COW updates. Unfortunately, this is the case with newly created file systems. As files are created and subsequently deleted in a newly created Btrfs file system, the metadata remains localized to the ends of the allocated nodes, thereby constantly getting overwritten in the process. This does not allow newly created nodes to have any remnant of deleted files as *Orphan-Items*. At the same time, the degree to which deallocated nodes containing deleted-file metadata are scattered in a fresh file system is significantly less due to low frequency of *redistribution* and *merging*, which further diminishes the recovery potential of newly created Btrfs volume. From this, we infer that the chances of recovering a deleted file on a newly created Btrfs volume are low, and whatever is recoverable remains insignificant given the magnitude of the data originally deleted.

Probability of evidence-extraction increases as Btrfs ages

Btrfs is one of the few file systems that can be regarded as the *Holy Grail* of forensic evidence as it supports forensically rich data structures due to its COW update model. However, such forensic richness does not come abruptly, but gradually as updates are performed, thereby scattering data all over the file system as a consequence of COW. We have already established that recovery on a freshly created Btrfs volume is difficult, as the data remains significantly less scattered. On the contrary, we can safely argue that the recovery potential of Btrfs increases significantly as the file system ages. Apart from ensuring that multiple copies of data exist in unallocated space of the file system as a result of COW updates, the frequency of valid `btrfs_items` among *Orphan-Items* also shows a gradual increase as nodes are merged and redistributed. This implies that even though deleted file recovery may be difficult initially, subsequent updates to file system data gradually increase the probability of recovery as the file system ages.

Internal nodes are the richest sources of forensic data

One of the key features of Btrfs is that its on-disk layout is a forest of B-trees with internal nodes acting as pointers and leaf nodes containing actual data. The internal nodes are simply flat lists of `[key, block-pointer]` pairs, which are packed into the front of node. Unlike leaf nodes, the remaining space in internal nodes is regarded as block slack space and remains largely sparse. In addition, the file system uses same block sizes for both internal and leaf nodes with a default block size of 4 KB. Furthermore, the file system performs block allocation to leaf and internal nodes at random without following a standard uniform distribution. This implies that a block allocated previously may be reallocated again in subsequent updates. This makes internal nodes a major source of digital evidence during forensic investigation. Since a major portion of internal nodes remains unused, a significant amount of forensic evidence can be recovered if the node previously happened to be a leaf node. Consequently, it also implies that if deleted data is not recovered

immediately, there is a higher possibility of recovering such data in subsequent future updates. Thus, internal nodes can serve as major repositories of deleted data and significantly enrich the forensic evidence collection during post-breach analysis.

Degree of retrieval depends upon whether nodes are merged or redistributed

Unlike other file system, Btrfs represents a paradigm shift not only in file system design but in terms of recovery as well. Btrfs exhibits a peculiar process of *merging* and *redistribution* on data deletion. These processes are responsible for balancing the Btrfs's B-trees and may ripple all the way up to the root. Since the minimum element constraint does not apply to root nodes, an unbalanced root is not a problem. The deletion operation unbalances the entire tree in specific scenarios such as the one wherein the data to be deleted is represented by 10 entries (`btrfs_items`) and the B-tree satisfies following conditions:

- Has at-least 10 leaf nodes.
- All nodes are at-most half full, that is removing at-least one entry unbalances the node.
- The entries of data to be deleted are distributed evenly over all the leaves of B-tree.

Since all leaf nodes are at-most half full initially, the deletion operation puts every leaf in an un-balanced state and hence none of the nodes is able to spare any entry that may help its neighbour to balance itself. This implies that the only way to balance the tree is to merge neighbouring nodes. On the contrary, in a scenario where the B-tree satisfies following conditions, only the last leaf of the B-tree gets un-balanced after deletion.

- Has at-least 10 leaf nodes.
- All nodes are filled to capacity, except for the last leaf which is at-most half full.
- The entries of data to be deleted are all contained in the last leaf node.

Since all other nodes are filled to their capacity, it is impossible to merge the rest of its entries into its neighbours. However, in order to balance the tree, entries are moved or redistributed from neighbouring nodes in such a way that the final configuration results in a balanced B-tree.

Merging and *redistribution* significantly affect the recovery potential of Btrfs. This is because, whenever the nodes are merged as a consequence of delete operation, the block slack spaces containing deleted content are overwritten and hence, the chances of recovering such data in its entirety is almost impossible. However, it may be possible to recover only a small fraction of deleted data. This implies that the degree of recoverability is least when nodes are merged. On the contrary, *redistribution* of nodes can prove a boon to forensic investigators during file recovery as the number of entries moved across nodes is significantly less. Consequently, the deleted content remains persistent and available for extraction. Thus, *redistribution* ensures maximum degree of recoverability and the highest possibility of recovering the data in its entirety.

Files with size less than 1 KB and greater than 4 KB have highest chances of recovery

B-tree file system stores file data either in an *inline-extent* or *regular-extent*. Btrfs supports a block size of 4 KB by default, and stores files with size less than 4 KB within leaf nodes as usual `btrfs_items`. Although it may not be possible to recover all the

files stored in *inline-extents*, but the chances of recovery increase with decreasing file sizes. This implies that smaller files have higher chances of recovery as they are less likely to be overwritten during node-balancing, and hence can be easily recovered from *Orphan-Items*. Furthermore, deleting files with size less than 1 KB are most likely to result in *redistribution* of node entries rather than *merging*.

On the other hand, larger files greater than 4 KB are stored using *regular-extents* outside leaf nodes. Although recovering a *regular-extent* seems relatively difficult (as data is stored outside the B-tree), the recoverability is high as *btrfs_file_extent_items* are relatively small sized in case of *regular-extents*. This significantly increase their chances to be recovered from *Orphan-Items*. In addition, random byte corruptions in *Orphan-Items* does not affect recovery as both *[disk block, length]* pair and *[logical file offset, length]* pair in *btrfs_file_extent_item* contain redundant mapping information, and hence recovering only one of them is enough to trace the *extent* back to its actual data. Furthermore, the Btrfs file system can be configured to support block sizes of 8 KB, 16 KB and so on. In such cases, file sizes greater than the block size and less than 1/4th of the block size will exhibit higher chances of recovery. On the other hand, file sizes that fall in between will exhibit relatively lower recovery chances.

Files with size 2–4 KB have least chances of recovery

If a deleted file falls in the range of 2–4 KB, the chances of its recovery are least due to two reasons. First, a 2–4 KB file is stored in an *inline-extent* that approximately accounts for about 50–100% of the total space available per leaf block. As a result, deleting such a file will certainly cause the nodes to be merged rather than *redistributed*. Second, it is impossible to recover such a file from *Orphan-Items* as *merging* causes loss of significant amounts of data even in case of small files.

Large files and fragmentation

The maximum theoretical limit on the size of individual files in Btrfs is 16 exabytes (EB) but practically it is limited to 7 EB. As Btrfs stores large files in *regular-extents*, the file system heavily relies on free and contiguous *extents* in order to maintain an even performance. Nevertheless, fragmentation occurs when the file system is low on such free and contiguous *extents*. However, Btrfs file system deals with fragmentation gracefully by employing batch processing and online-defragmentation (Rodeh et al., 2013). The file system accumulates updates in memory and writes them to disk at sync time. This helps in reducing fragmentation of files by allowing the file system to perform batch optimizations, like delayed allocation and collocating data from same files. However, given the sync interval of 30 seconds (Rodeh et al., 2013) and limited buffer space, the batch processing solution is not sufficient to combat disk fragmentation. As a means of last resort, the file system makes use of *reloc-tree* to relocate scattered *regular-extents*. However, this relocation is done on chunk-by-chunk basis by identifying all the related-scattered *regular-extents* using *extent-tree*, and copying these *regular-extents* to newer-contiguous locations.

Due to this relocation, the probability of overwriting disk areas that may contain *regular-extents* pertaining to other deleted files does exist. However, the proposed evidence-extraction procedure relies on the use of *Orphan-Items*, and these *items* are not part of *regular-extents*. Therefore, relocation of large files does not overwrite *Orphan-Items* but the *regular-extents* referenced by them may be overwritten. Nevertheless, Btrfs always uses relocation based defragmentation as a last resort when other optimizations like collocation of related data and its delayed allocation have failed. Also, large files are generally read-only (Bhat, 2012), leaving very

little chances for relocation based defragmentation to creep in during file operations.

Challenges

Compression

Btrfs supports transparent file compression at *extent-level* using two algorithms (Rodeh et al., 2013) – ZLIB and LZO. Also, compression is on per-file basis. This implies that a single Btrfs mount point can have some files that are uncompressed, some that are compressed with LZO, and some with ZLIB. This makes Btrfs's compression technique very challenging, and current digital forensic techniques are unable to deal with such a significant mix of compression states – not compressed, compressed, and compressed with a variety of algorithms (Beebe et al., 2009). Furthermore, as the file system is still under development, work is being done for the support of other compression algorithms as well. Currently, work is in progress to add LZ4. Snappy support which compresses slower than LZO but decompresses much faster has also been proposed.

Encryption

Although Btrfs does not provide built-in encryption functionality yet, it works safely with partition encryption techniques like *dm_crypt*. *dm_crypt* is a transparent disk encryption that supports full disk encryption (FDE) and *pre-boot authentication* that prevents access to any data unless the necessary credentials are supplied. This implies that without the necessary credentials to unlock the drive, it is impossible for digital investigators to get access to any logical level information (Casey and Stallatos, 2008). In addition, use of stackable file system (like *ecryptfs* (Halcrow, 2005)) with Btrfs can further complicate forensic investigation due to transparent file encryption. Furthermore, recovery of encrypted data can be challenging from a legal viewpoint as well (Casey, 2011).

Solid state drive support

Btrfs is *SSD-aware* (Leventhal, 2013) enabling it to avoid write amplification and unnecessary seek optimizations, and provide *TRIM* (Hyun et al., 2011) support to the underlying garbage collector of the SSD. However, SSDs have introduced many dramatic challenges to the principles of computer forensics including destroying evidence catastrophically under their own volition (Bell and Boddington, 2010). As SSDs are heading towards replacing hard disks, native support for TRIM and other SSD optimizations by Btrfs make it the file system of choice for SSDs. This implies that the challenges posed by SSDs are only amplified to an exponential scale with Btrfs. Furthermore, the firmware built into many (and possibly all) of these drives allows them to destroy data simply by being powered on, even when not connected to a PC or under the apparent control of an operating system. Studies have shown that current digital forensic methods are futile when it comes to SSDs (Bell and Boddington, 2010).

Conclusion

We identified forensically important data structures of Btrfs, analysed the impact of node *splitting*, *redistribution* and *merging* on the identified forensic artifacts during normal file and directory operations, proposed an evidence-extraction procedure based on the identified forensic artifacts and observed file system state-change, and discussed the results. The findings suggest that evidence-extraction in a fresh B-tree file system is difficult as the

data and metadata remain localized to the ends of the allocated nodes, and thus are constantly overwritten in the process. However, scattering of data (due to COW) all over the file system happens as the file system age, which increases the probability of evidence-extraction. The findings also point towards internal nodes as being the richest sources of forensic data, as a major portion of these nodes remains unused, and thus may contain data from previous allocations. Also, the node-balancing mechanism of Btrfs play an important role in evidence-extraction. Depending upon whether nodes are merged or redistributed, the degree of extraction varies. Consequently, files with size less than 1 KB and greater than 4 KB have highest chances of recovery, and files with size 3–4 KB have least chances of recovery. However, node-balancing is not invoked in most of the metadata operations and thus such operations preserve the forensic data. Finally, we alerted forensic community about the unique challenges that compression, encryption and SSD's present to a forensic investigator in Btrfs.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.dib.2018.04.100>.

References

- Barik, M.S., Gupta, G., Sinha, S., Mishra, A., Mazumdar, C., 2007. An efficient technique for enhancing forensic capabilities of ext2 file system. *Digit. Invest.* 4, 55–61.
- Beebe, N.L., Stacy, S.D., Stuckey, D., 2009. Digital forensic implications of zfs. *Digit. Invest.* 6, S99–S107.
- Bell, G.B., Boddington, R., 2010. Solid state drives: the beginning of the end for current practice in digital forensic recovery? *The Journal of Digital Forensics, Security and Law: JDFSL* 5 (3), 5.
- Bhat, W.A., 2012. Design Considerations for Developing a Disk File System. PhD thesis. University of Kashmir.
- Bhat, W.A., Quadri, S., 2011. A quick review of on-disk layout of some popular disk file systems. *Global J. Comput. Sci. Technol.* 11 (6), 1–17.
- Bhat, W.A., Quadri, S.M.K., 2012. After-deletion data recovery: myths and solutions. *Comput. Fraud Secur.* 2012 (4), 17–20.
- Bhat, W.A., Quadri, S., 2012. Restfs: secure data deletion using reliable & efficient stackable file system. In: *Applied Machine Intelligence and Informatics (SAMI), 2012 IEEE 10th International Symposium on*. IEEE, pp. 457–462.
- Bhat, W.A., Quadri, S.M.K., 2013. Poster: Dr. watson provides data for post-breach analysis. In: *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. ACM, pp. 1445–1448.
- Casey, E., 2011. *Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet*. Academic press.
- Casey, E., Stellatos, G.J., 2008. The impact of full disk encryption on digital forensics. *ACM SIGOPS - Oper. Syst. Rev.* 42 (3), 93–98.
- Chen, W., Liu, C.-m., 2010. The analysis and design of linux file system based on computer forensic. In: *Computer Design and Applications (ICDDA), IEEE, 2010 International Conference on*, vol. 2, pp. 2–60.
- Comer, D., 1979. Ubiquitous b-tree. *ACM Comput. Surv.* 11 (2), 121–137.
- Fairbanks, K.D., 2012. An analysis of ext4 for digital forensics. *Digit. Invest.* 9, S118–S130.
- Halcrow, M.A., 2005. Ecryptfs: an enterprise-class encrypted filesystem for linux. In: *Proceedings of the 2005 Linux Symposium*, vol. 1, pp. 201–218.
- Hyun, C., Choi, J., Lee, D., Noh, S.H., 2011. "To trim or not to trim: judicious trimming for solid state drives. In: *Poster Presentation in the 23rd ACM Symp. On Operating Systems Principles (SOSP)*, Citeseer.
- Leventhal, A.H., 2013. A file system all its own. *Commun. ACM* 56 (5), 64–67.
- Nagar, R., 1997. *Windows NT File System Internals: a Developer's Guide*, vol. 786. O'Reilly.
- Newsham, T., Palmer, C., Stamos, A., Burns, J., 2007. Breaking forensics software: weaknesses in critical evidence collection. In: *Proceedings of the 2007 Black Hat Conference*.
- Park, Y., Chang, H., Shon, T., 2015. Data Investigation Based on Xfs File System Metadata. *Multimedia Tools and Applications*, pp. 1–23.
- Piper, S., Davis, M., Manes, G., Shenoi, S., 2005. Detecting hidden data in ext2/ext3 file systems. In: *Advances in Digital Forensics*. Springer, pp. 245–256.
- Powell, H., 2012. Zfs and btrfs: a quick introduction to modern filesystems. *Linux J.* 2012 (218), 5.
- Rodeh, O., 2008. B-trees, shadowing, and clones. *ACM Trans. Storage* 3 (4), 2.
- Rodeh, O., 2010. Deferred Reference Counters for Copy-On-Write b-Trees. Technical Report rj10464. IBM.
- Rodeh, O., Bacik, J., Mason, C., 2013. Btrfs: the linux b-tree filesystem. *ACM Trans. Storage* 9 (3), 9.
- Wikipedia, "stat (system call)." [https://en.wikipedia.org/wiki/Stat_\(system_call\)](https://en.wikipedia.org/wiki/Stat_(system_call)). [online; accessed on 21-June-2018].